

GUIA DE ESTUDIO

Tests en JUnit 5, explicados desde cero

RestoManager

Una guía corta para entender qué se prueba, por qué se usan mocks y cómo leer una ejecución de Maven.

IDEA CENTRAL

Un test prepara una situación, ejecuta una acción y comprueba si el resultado es el esperado.

Basada en la suite real del Backend: 44 pruebas unitarias, 39 de integración y un guardrail para proteger la base de datos.

1. La idea central

Antes de memorizar anotaciones, entendí qué pregunta responde cada prueba.

Un test es una pregunta con una respuesta verificable.

Ejemplo: "Si intento descontar 2 unidades de un producto con stock 5, ¿el stock queda en 3 y se llama al DAO para guardar el cambio?"

PREPARAR Creo datos, mocks y respuestas controladas.	EJECUTAR Llamo una operación concreta del sistema.	COMPROBAR Comparo el resultado y las colaboraciones importantes.
--	--	--

En el proyecto, esta secuencia aparece en servicios como ProductoService, MesaService y PedidoService. El objetivo es comprobar reglas de negocio sin depender de una pantalla.

Dos niveles de prueba

Nivel	Pregunta simple	Qué usa
Unitario	¿La regla funciona por sí sola?	Objetos en memoria y mocks. No abre la base.
Integración	¿Las piezas reales trabajan juntas?	DAO real, JDBC, SQL y restomanager_test.

La suite completa del Backend reúne 83 casos: 44 unitarios ejecutados por Surefire y 39 integraciones ejecutadas por Failsafe.

2. Tests unitarios

Un unitario aísla una regla para que el fallo tenga una causa fácil de encontrar.

Pensalo como probar una pieza sobre la mesa, sin conectar todo el restaurante. PedidoService puede necesitar un PedidoDAO, un MesaService y un ProductoService. En el test, esos colaboradores se reemplazan por mocks.

Mock = colaborador controlado

El mock no es el DAO real. Es un objeto que responde exactamente lo que el test necesita. Así la prueba se concentra en la regla del servicio.

Fragmento didáctico basado en ProductoServiceTest.java

```
@Test
void descontarStock_actualizaLaCantidad() {
    // Arrange: preparo la respuesta del mock
    when(dao.getProductoPorId(1)).thenReturn(producto);
    when(dao.editar(producto))
        .thenReturn("Producto editado correctamente");

    // Act: ejecuto una sola accion
    String resultado = service.descontarStock(1, 2);

    // Assert: compruebo resultado, estado y colaboracion
    assertEquals("Producto editado correctamente", resultado);
    assertEquals(3, producto.getStock());
    verify(dao).editar(producto);
}
```

Parte	Qué ocurre
Arrange	El mock devuelve un producto con stock 5 y el DAO acepta la edición.
Act	El servicio descuenta 2 unidades.
Assert	El resultado es correcto, el stock queda en 3 y el DAO fue llamado.

Si la prueba falla, sabés dónde mirar: la regla del servicio, el estado del producto o la colaboración con el DAO.

3. JUnit 5 y Mockito sin misterio

Las anotaciones describen cómo preparar y ejecutar cada caso.

Elemento	En palabras simples
@Test	Marca un método como caso de prueba.
@BeforeEach	Ejecuta una preparación antes de cada caso. Reinicia el escenario.
@ExtendWith(MockitoExtension.class)	Activa Mockito para crear e inyectar mocks.
@Mock	Declara un colaborador falso y controlable.
when(...).thenReturn(...)	Le dice al mock qué respuesta debe dar.
assertEquals / assertTrue	Comprueba un valor o una condición.
assertNull / assertNotNull	Comprueba ausencia o existencia de un objeto.
verify(dao).metodo()	Comprueba que una colaboración ocurrió.
never / verifyNoInteractions	Comprueba que una colaboración no ocurrió.

La diferencia que más confunde

Código	Pregunta que responde
when(dao.buscar(1)).thenReturn(producto)	¿Qué debe devolver el colaborador cuando lo consulte?
assertEquals(3, producto.getStock())	¿El estado final tiene el valor esperado?
verify(dao).editar(producto)	¿El servicio pidió guardar el cambio?
verify(dao, never()).editar(any())	¿El servicio evitó guardar cuando había un error?

La preparación real de PedidoServiceTest usa inyección por constructor.

```
@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {
    @Mock private PedidoDAO pedidoDAO;
    @Mock private MesaService mesaService;
    private PedidoService service;

    @BeforeEach
    void setUp() {
        service = new PedidoService(pedidoDAO, mesaService, productoService);
    }
}
```

4. Tests de integración

Acá no se simula la persistencia: se comprueba el camino real hasta la base de prueba.

Un test de integración responde otra pregunta. No alcanza con saber que el servicio llama a un DAO. También hay que comprobar que el SQL, las tablas, las claves y el mapeo JDBC funcionan juntos.

CASO Creo datos válidos para el escenario.	JDBC El DAO ejecuta SQL real contra MariaDB.	EVIDENCIA Leo de nuevo y compruebo el dato guardado.
---	---	---

Fragmento resumido basado en ProductoDAOImplIT.java

```
@Test
void insertar_productoValido_debePoderseLeerPorId() {
    Producto producto = crearProductoPrueba("TestProducto");

    // Act: INSERT real
    dao.insertar(producto);
    productoInsertadoId = producto.getIdProducto();

    // Assert: lectura real despues del INSERT
    Producto leido = dao.getProductoPorId(productoInsertadoId);
    assertNotNull(leido);
    assertEquals("TestProducto", leido.getNombre());
}
```

Por qué se llaman IT

Las clases terminadas en IT.java se reservan para integración. Maven las ejecuta con Failsafe bajo el perfil integration-tests. Las clases Test.java quedan para Surefire y los unitarios.

La base segura es parte del test

DedicatedDatabaseExtension revisa db.url antes de cada clase de integración. Si la URL no termina en restomanager_test, la suite se detiene antes de abrir conexiones.

Guardrail real del proyecto, simplificado para estudiar.

```
String url = System.getProperty("db.url", "");
if (!url.matches("(?i).*[/:]restomanager_test(?:[?;].*)?$")) {
    throw new IllegalStateException(
        "Las integraciones requieren restomanager_test");
}
```

5. Cómo se ejecutan

Maven separa las pruebas por propósito y deja una salida que se puede leer.

No confundas compilar con probar. Un BUILD SUCCESS solo demuestra la ejecución completa del comando. Si el log dice Tests run: 44, Failures: 0, Errors: 0, entonces sí hay evidencia de que los casos pasaron.

Comandos documentados para el proyecto.

```
# Unitarias: rapidas y sin base de datos
mvn -pl Backend clean verify

# Integracion: MariaDB local en restomanager_test
mvn -pl Backend -Pintegration-tests clean verify \
  -Ddb.url=jdbc:mysql://127.0.0.1:3307/restomanager_test \
  -Ddb.user=root -Ddb.password=
```

Herramienta	Qué hace
JUnit Jupiter 5.10.1	Da las anotaciones y las aserciones.
Mockito 5.23.0	Reemplaza colaboradores para aislar unitarios.
Surefire 3.5.4	Ejecuta clases *Test.java.
Failsafe 3.5.4	Ejecuta clases *IT.java con el perfil de integración.
JaCoCo 0.8.15	Mide líneas, ramas e instrucciones cubiertas.

Salida final documentada

Unitarias: 44/44 exitosas. Integración: 39/39 exitosas. Total: 83 pruebas sin fallos, errores ni omisiones.

Para encontrar el detalle de cobertura, abrí Backend/target/site/jacoco/index.html después de ejecutar Maven.

6. Cómo leer los resultados

Los números sirven para orientar la conversación, no para reemplazar la comprensión.

Resultado	Qué significa
Tests run: 44	Se ejecutaron 44 casos unitarios.
Failures: 1	Un assert no coincidió con lo esperado. La lógica respondió algo distinto.
Errors: 1	El test no pudo terminar por una excepción o problema de configuración.
Skipped: 1	El caso fue omitido y no aporta evidencia en esa corrida.
BUILD SUCCESS	El comando terminó bien. Hay que leer también el resumen de tests.

Cobertura: qué responde y qué no responde

Dato del proyecto	Lectura simple
77,5% de líneas	Casi 8 de cada 10 líneas ejecutables fueron recorridas por los casos.
56,6% de ramas	Un poco más de la mitad de las decisiones tomó sus caminos posibles.
78,6% de instrucciones	La mayoría de las instrucciones registró al menos una ejecución.

Cobertura no significa calidad automática

Un test puede ejecutar una línea sin comprobar un resultado útil. La pregunta correcta es: ¿qué comportamiento importante queda protegido?

Distribución real de la suite

Grupo	Clases	Casos
Servicios, modelos y controladores	7 unitarias	44
DAOs y flujo JDBC	7 integraciones	39
Protección de base	1 extensión sin @Test propio	Guardrail

7. Cómo explicarlo en una exposición

Usá una estructura fija y corta. No hace falta leer todo el archivo.

Plantilla de 30 segundos

“Este test verifica [comportamiento]. En Arrange preparo [datos y mocks]. En Act llamo [método]. En Assert compruebo [resultado y colaboración]. Es unitario porque no usa una base real.”

Ejemplo aplicado a PedidoServiceTest

Momento	Qué podés decir
Qué protege	La creación de un pedido calcula el total, descuenta stock y ocupa la mesa.
Arrange	El mock devuelve una mesa libre, un producto con precio y respuestas exitosas.
Act	Llamo crearPedido con usuario, mesa y detalle.
Assert	Compruebo el total, el estado ABIERTO, el descuento de stock y la llamada a ocupar.
Por qué es unitario	PedidoDAO, MesaService y ProductoService están controlados por Mockito.

Si te preguntan por integración

Respondé: “Ahí se reemplaza el mock por el DAO real. El test ejecuta JDBC, SQL y lecturas contra restomanager_test. Después limpia los datos para que el siguiente caso empiece limpio.”

Tres frases que conviene recordar

UNITARIO Prueba una regla aislada.	MOCK Controla una dependencia.	INTEGRACION Prueba piezas reales conectadas.
--	--	--

8. Hoja de repaso

Leé la pregunta de la izquierda e intentá responder antes de mirar la explicación.

Pregunta	Respuesta corta
¿Qué es un test?	Una pregunta automática con una condición esperada.
¿Qué es Arrange?	Preparar datos, objetos y respuestas de mocks.
¿Qué es Act?	Ejecutar la operación que quiero evaluar.
¿Qué es Assert?	Comprobar resultado, estado o condición.
¿Para qué sirve when?	Para preparar la respuesta de un mock.
¿Para qué sirve verify?	Para comprobar que ocurrió una colaboración.
¿Por qué usar Mockito?	Para aislar la regla y evitar dependencias reales.
¿Qué diferencia hay con IT?	IT conecta DAO, JDBC, SQL y base de prueba reales.
¿Qué hace DedicatedDatabaseExtension?	Bloquea integraciones si la URL no apunta a restomanager_test.
¿Qué mide JaCoCo?	Qué parte del código fue recorrida, no si todo está bien probado.

Mini práctica

1. Si un test usa `when(dao.buscar()).thenReturn(...)`, ¿está preparando o comprobando? 2. Si usa `verify(dao).editar(...)`, ¿qué está comprobando? 3. Si la clase termina en `IT`, ¿esperás una base real o un mock?

Respuestas

1. Está preparando. 2. Está comprobando una colaboración. 3. Una integración con dependencias reales y la base dedicada.

Resumen final

Si ves...	Pensá...
@Mock + when	La dependencia está controlada.
assert...	Estoy comprobando un resultado o estado.
verify...	Estoy comprobando una colaboración.
*IT.java + db.url	Estoy ante una integración real y debo revisar el aislamiento.